



NORTH SOUTH UNIVERSITY

Department of Electrical & Computer Engineering

AIris: A Purpose-Built Assistive AI for Navigation and Interaction

**CSE 499A Senior Capstone Design Project
Final Report**

Authors:

Adib Ar Rahman Khan

Student ID: 2212708042

Saumik Saha Kabbya

Student ID: 2211204042

Supervisor:

Dr. Mohammad Rashedur
Rahman

Associate Professor

Department of ECE

September 2, 2025

Abstract

For millions of visually impaired individuals worldwide, navigating environments presents constant challenges that current assistive technologies only partially address. Existing solutions suffer from high latency, dependency on unreliable cloud connections, and a critical lack of contextual understanding, providing simple object labels rather than actionable environmental awareness. This paper presents AIris, a novel wearable AI system designed to overcome these fundamental limitations through a hybrid, two-stage software architecture that combines local vision processing with intelligent reasoning. Our core innovation lies in task-specific prompt engineering that transforms generic Large Language Models (LLMs) into specialized assistive experts. Through rigorous evaluation using custom datasets representative of real-world assistive scenarios, we demonstrate both the viability of our architectural approach and identify critical challenges in latency (12.19s average) and safety-critical reasoning. While achieving 87% helpfulness in indoor navigation tasks, the system's performance in dynamic hazard detection (40% helpfulness, complete safety failure) reveals fundamental areas requiring optimization. This work establishes a validated software foundation and provides a data-driven roadmap for hardware integration and deployment optimization in subsequent development phases.

Keywords: Assistive Technology, Computer Vision, Natural Language Processing, Wearable Computing, Accessibility, Human-Computer Interaction

Contents

1	Introduction	6
1.1	Background and Motivation	6
1.2	Problem Statement	6
1.3	Research Objectives	7
2	Literature Review	7
2.1	Assistive Technology Landscape	7
2.2	Vision-Language Models in Assistive Applications	7
2.3	Edge Computing for Assistive Devices	8
2.4	Research Gap Identification	8
3	System Design and Architecture	8
3.1	Conceptual Framework	9
3.2	Hardware Architecture	9
3.3	Software Architecture	9
3.4	Processing Pipeline	10
4	Methodology and Implementation	11
4.1	Development Methodology	11
4.1.1	Phase 1: Strategic Focus Definition	11
4.1.2	Phase 2: Architecture Implementation	11
4.2	Prompt Engineering Innovation	12
4.2.1	Iterative Prompt Refinement Process	12
4.2.2	Prompt Template Design	12
4.3	Evaluation Framework Development	13
4.3.1	Custom Dataset Creation	13
4.3.2	Evaluation Metrics	13
5	Demo System and Software Interface	13
5.1	Core Testing System Design	13
5.2	Evaluation Dashboard Interface	14
5.3	Processing Pipeline Visualization	14
5.4	Ground Truth Comparison System	15
5.5	Mode-Specific Testing Capabilities	16
5.6	Performance Metrics Dashboard	17
5.7	Technical Implementation Features	18
5.7.1	One-Click Example Testing	18

5.7.2	Configurable Analysis Parameters	18
5.7.3	Export and Logging Capabilities	18
5.7.4	Error Visualization and Debugging	18
6	Results and Analysis	19
6.1	Experimental Setup	19
6.2	Performance Results	19
6.3	Performance Analysis by Test Case	19
6.3.1	Indoor Navigation Success	20
6.3.2	Object Finding Limitations	20
6.3.3	Critical Safety Failure	21
6.4	Latency Analysis	21
6.5	Multi-Dimensional Performance Assessment	22
6.6	Safety-Critical Analysis	22
7	Discussion	23
7.1	Architectural Validation	23
7.2	Critical Limitations Identified	24
7.2.1	Latency Crisis	24
7.2.2	Spatial Reasoning Deficiencies	24
7.2.3	Safety-Critical Failures	24
7.3	Implications for Future Development	24
7.4	Broader Research Contributions	25
8	Future Work and Development Roadmap	25
8.1	CSE 499B Development Priorities	25
8.1.1	Priority 1: Aggressive Latency Optimization	26
8.1.2	Priority 2: Enhanced Safety and Reliability	26
8.1.3	Priority 3: Hardware Integration and Optimization	27
8.1.4	Priority 4: User-Centered Validation	27
8.2	Long-Term Research Directions	27
9	Conclusion	27
9.1	Key Achievements	28
9.2	Critical Findings	28
9.3	Research Impact	29
9.4	Path Forward	29
A	Core Assistive Use Cases	31
A.1	Scenario 1: Indoor Navigation & Obstacle Avoidance	31

A.2	Scenario 2: Object Localization & Interaction	31
A.3	Scenario 3: Environmental Awareness	31
A.4	Scenario 4: Dynamic Hazard Detection	32
B	System Prompts and Templates	32
B.1	Navigation Mode Prompt Template	32
B.2	Object Finding Mode Prompt Template	33
B.3	Hazard Detection Mode Prompt Template	34
C	Technical Implementation Details	34
C.1	Development Environment Configuration	34
C.2	Performance Optimization Strategies	35
C.2.1	Frame Selection Algorithm	35
C.2.2	Batch Processing Optimization	36
C.3	Error Handling and Robustness	36

List of Figures

1	AIris System Architecture Overview showing component distribution and processing pipeline flow	10
2	AIris Evaluation Dashboard Main Interface showing video upload, mode selection, and configuration controls	14
3	Processing Pipeline Visualization showing Stage 1 raw computer vision outputs and Stage 2 LLM-processed results	15
4	Ground Truth Comparison Interface showing direct comparison between AI output and ideal responses with performance metrics	16
5	Assistive Mode Selection Interface enabling testing across different specialized prompt templates and evaluation scenarios	17
6	Performance Metrics Dashboard displaying real-time system performance data and resource utilization statistics	18
7	AIris System Performance by Test Case showing helpfulness scores across evaluation scenarios	20
8	Latency Analysis showing individual test case performance and comparison with target requirements	21
9	Multi-Metric Performance Dashboard showing normalized scores across all evaluation dimensions	22
10	Task Success vs Safety Analysis with quadrant-based risk assessment zones	23
11	Development Progress comparison between CSE 499A achievements and CSE 499B targets	26

1 Introduction

1.1 Background and Motivation

According to the World Health Organization, at least 2.2 billion people globally have a near or distance vision impairment [1]. For this vast population, tasks that sighted individuals take for granted—navigating a room, locating objects, or assessing environmental hazards—represent daily challenges that significantly impact independence and quality of life. While traditional mobility aids such as the white cane and guide dogs provide invaluable assistance, they offer limited information about the broader environmental context that sighted individuals naturally perceive.

The emergence of artificial intelligence, particularly in computer vision and natural language processing, presents unprecedented opportunities to develop more sophisticated assistive technologies. However, current solutions remain fragmented, often providing technical capabilities without addressing the specific needs and constraints of visually impaired users. This disconnect between technological capability and user requirements motivates our research into purpose-built assistive AI systems.

1.2 Problem Statement

Contemporary visual assistance solutions, primarily smartphone applications and specialized devices, face several critical limitations that prevent their widespread adoption and effectiveness:

- **Latency Issues:** Cloud-dependent processing introduces delays of 5-10 seconds, making real-time navigation and hazard avoidance impractical
- **Connectivity Dependency:** Intermittent or unavailable internet connections render intelligent features inoperative, creating unreliable user experiences
- **Contextual Understanding Gap:** Systems excel at object identification but fail to synthesize information into coherent, actionable scene understanding
- **Interface Complexity:** Solutions confined to smartphone interfaces present usability challenges for users who need hands-free operation
- **Cost Barriers:** Proprietary hardware solutions often carry prohibitive costs, limiting accessibility for users who would benefit most

1.3 Research Objectives

This research addresses these limitations through the development of AIris, a wearable AI system specifically designed for assistive navigation and environmental interaction. Our primary objectives include:

1. Design and implement a hybrid architecture combining local vision processing with cloud-based reasoning
2. Develop task-specific prompt engineering methodologies to optimize LLM performance for assistive applications
3. Create comprehensive evaluation frameworks for assessing assistive AI system performance
4. Establish performance baselines and identify optimization priorities for hardware integration
5. Validate the feasibility of real-time, contextual scene description for navigation assistance

2 Literature Review

2.1 Assistive Technology Landscape

The field of assistive technology for visual impairment has evolved significantly with advances in sensor technology and artificial intelligence. Elmannai and Elleithy [2] provide a comprehensive survey of sensor-based assistive devices, highlighting the transition from simple obstacle detection systems to more sophisticated computer vision-based approaches. Their analysis reveals persistent challenges in battery life, processing power, and user interface design that continue to limit practical deployment.

Recent work by Naayini et al. [3] specifically examines AI-powered assistive technologies, documenting the emergence of machine learning approaches for object recognition, scene understanding, and navigation assistance. Their findings confirm the potential of AI-driven solutions while emphasizing the gap between research prototypes and deployable systems suitable for daily use.

2.2 Vision-Language Models in Assistive Applications

The development of Vision-Language Models (VLMs) has created new possibilities for assistive technology applications. Foundational work on BLIP-2 by Li et al. [4] demonstrates the capability of these models to generate detailed, contextually relevant descriptions of

visual scenes. Similarly, the LLaVA architecture presented by Liu et al. [5] shows promise for instruction-following visual reasoning tasks.

However, these general-purpose models require significant adaptation for assistive applications. The challenge lies not in their descriptive capability, but in their ability to provide concise, actionable, and safety-prioritized information suitable for real-time navigation assistance.

2.3 Edge Computing for Assistive Devices

The deployment of AI models on resource-constrained devices presents significant technical challenges. Wang and Wong [6] explore micro-architecture design considerations for computer vision applications on mobile devices, emphasizing the importance of model optimization and hybrid processing approaches.

Their work validates our architectural decision to implement a two-stage pipeline: performing lightweight vision processing locally while leveraging cloud-based services for complex reasoning tasks. This approach balances processing efficiency with the sophisticated reasoning capabilities required for contextual scene understanding.

2.4 Research Gap Identification

Our literature review reveals a critical gap between generic AI capabilities and specific assistive technology requirements. While current VLMs excel at comprehensive scene description, visually impaired users require:

- Concise, actionable information rather than verbose descriptions
- Safety-prioritized content highlighting potential hazards
- Spatial relationship information for navigation guidance
- Real-time processing capabilities for dynamic environments
- Reliable operation independent of internet connectivity

AIris addresses this gap through specialized prompt engineering and architectural design optimized for assistive applications rather than general-purpose vision-language tasks.

3 System Design and Architecture

3.1 Conceptual Framework

AIris is conceptualized as a wearable AI companion providing instant, contextual scene descriptions for visually impaired users. The system prioritizes simplicity of interaction through a single-button interface while delivering sophisticated environmental awareness through advanced AI processing.

The physical design separates functionality across two components for optimal user comfort and system modularity:

- **Spectacle Unit:** Lightweight glasses-mounted component containing the camera and directional speaker
- **Pocket Unit:** Portable processing unit housing the Raspberry Pi 5, battery, and user input controls

This separation minimizes the weight and obtrusiveness of the head-mounted component while centralizing processing power in a more conventional portable form factor.

3.2 Hardware Architecture

Table 1 presents the complete hardware specification for the AIris system. Component selection prioritizes processing capability, power efficiency, and cost-effectiveness to ensure practical deployment feasibility.

Table 1: AIris Hardware Architecture Specifications

Component	Specification	Purpose
Processing Unit	Raspberry Pi 5 (8GB RAM)	Core computing and AI processing
Camera Module	High-resolution USB/CSI	Real-time image capture
Audio Output	Directional mini-speaker	Private, non-intrusive feedback
Power Supply	10,000mAh+ battery pack	All-day portable operation
User Interface	Single tactile button	Simple system activation
Enclosure	Custom 3D-printed case	Protection and portability
Connectivity	Wi-Fi 6, Bluetooth 5.0	Cloud services and audio

3.3 Software Architecture

The AIris software architecture implements a modular, two-stage processing pipeline designed for maintainability and performance optimization. Figure 1 illustrates the complete system architecture and processing flow.

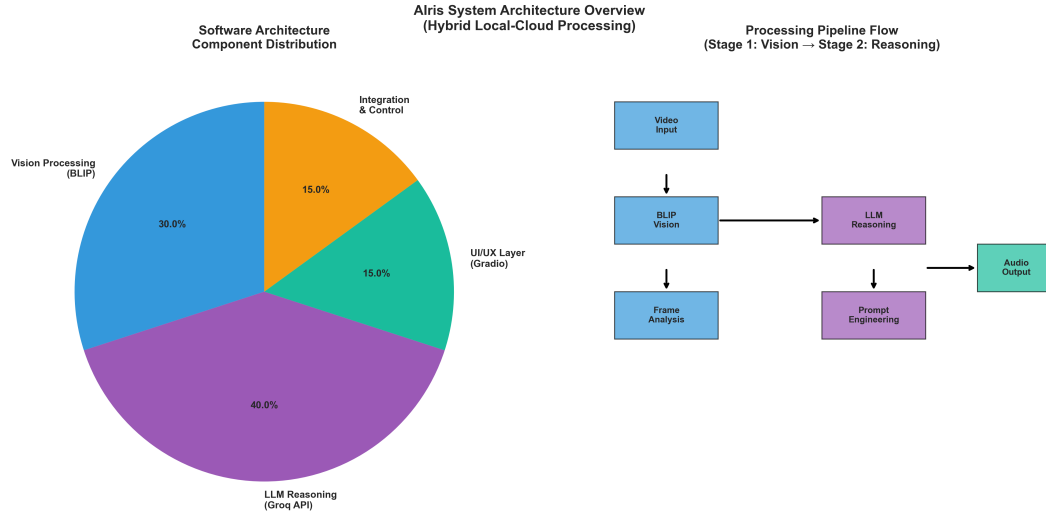


Figure 1: AIs System Architecture Overview showing component distribution and processing pipeline flow

The software architecture consists of four primary modules:

1. **Vision Processing Module (pipeline.py):** Handles all computer vision tasks using PyTorch and BLIP models for frame extraction and initial scene analysis
2. **Reasoning Engine (llm_integrations.py):** Manages communication with cloud-based LLM services for contextual interpretation and response generation
3. **Prompt Management System (prompts.py):** Contains task-specific prompt templates optimized for different assistive scenarios
4. **Application Controller (app.py):** Orchestrates data flow between modules and manages the user interface through Gradio

3.4 Processing Pipeline

The two-stage processing pipeline represents the core innovation of the AIs system:

Stage 1: Local Vision Analysis

- 1: Capture video frames from camera input
- 2: Extract key frames using temporal analysis
- 3: Process frames through BLIP model for object detection and scene description
- 4: Generate list of raw observational data

Stage 2: Contextual Reasoning

- 1: Select appropriate task-specific prompt template
- 2: Combine raw observations with contextual reasoning prompts
- 3: Submit to cloud-based LLM service (Groq API)

- 4: Receive synthesized, actionable scene description
- 5: Convert to audio output for user

This architecture ensures that computationally intensive vision processing occurs locally, minimizing latency for basic scene analysis, while leveraging cloud resources for sophisticated reasoning tasks that require large-scale language models.

4 Methodology and Implementation

4.1 Development Methodology

The AIris development process followed a systematic, phase-based approach designed to establish clear objectives and measurable outcomes at each stage.

4.1.1 Phase 1: Strategic Focus Definition

Rather than developing a generic video description system, we established four core assistive use cases to guide all subsequent technical decisions:

1. **Indoor Navigation & Obstacle Avoidance:** Providing spatial awareness and clear path identification
2. **Object Localization & Interaction:** Assisting users in finding and interacting with specific items
3. **Environmental Awareness:** Offering contextual understanding of new spaces and social situations
4. **Dynamic Hazard Detection:** Identifying and alerting users to moving obstacles or safety concerns

This use case definition ensured that every technical component serves a specific assistive function rather than pursuing general-purpose video analysis capabilities.

4.1.2 Phase 2: Architecture Implementation

Initial development occurred in a monolithic application structure to enable rapid prototyping and experimentation. However, to establish a robust foundation for future development, we refactored the system into the modular architecture described in Section 3.3.

This refactoring process included:

- Separation of vision processing, reasoning, and user interface concerns

- Implementation of standardized interfaces between modules
- Development of comprehensive error handling and logging systems
- Creation of configuration management for different deployment scenarios

4.2 Prompt Engineering Innovation

Our most significant technical contribution lies in the development of task-specific prompt engineering methodologies that transform general-purpose LLMs into specialized assistive experts.

4.2.1 Iterative Prompt Refinement Process

Initial experiments with simple prompts produced verbose, conversational outputs unsuitable for assistive applications:

Initial Prompt: "Summarize these events."

Typical Output: "In this scene, there appears to be a living room setting with what looks like various furniture pieces. The user should please be careful as there might be obstacles..."

Through systematic experimentation, we developed highly constrained, rule-based prompts that produce concise, actionable outputs:

Final Prompt Structure: Task-specific context + Strict formatting rules + Explicit negative constraints + Direct command structure

Optimized Output: "You are in a living room. Clear path ahead. Armchair to your right, coffee table to your left."

4.2.2 Prompt Template Design

The final prompt templates incorporate several key design principles:

- **Specialist Identity:** Establishing the AI as a visual assistance expert rather than a general conversationalist
- **Concrete Examples:** Providing ideal input-output pairs to guide response generation
- **Negative Constraints:** Explicitly prohibiting conversational filler and safety disclaimers
- **Format Enforcement:** Requiring specific response structures optimized for audio delivery
- **Brevity Requirements:** Limiting responses to 1-2 sentences maximum

4.3 Evaluation Framework Development

To ensure rigorous assessment of system performance, we developed a comprehensive evaluation framework incorporating both quantitative metrics and qualitative analysis.

4.3.1 Custom Dataset Creation

Rather than using generic computer vision datasets, we created a targeted evaluation dataset consisting of video scenarios specifically representing our core use cases. Each video includes:

- Realistic environmental conditions encountered by visually impaired users
- Multiple objects and spatial relationships requiring contextual interpretation
- Dynamic elements such as moving obstacles or changing lighting conditions
- Ground truth descriptions written from the perspective of an ideal assistive system

4.3.2 Evaluation Metrics

Our evaluation framework incorporates multiple performance dimensions:

- **Latency:** End-to-end processing time from input to audio output
- **Semantic Helpfulness:** Subjective assessment of description utility (0-1 scale)
- **Task Success Rate:** Binary measure of whether the description enables task completion
- **Safety Score:** Assessment of accurate hazard identification and directional guidance
- **Conciseness:** Evaluation of response brevity and actionability

5 Demo System and Software Interface

5.1 Core Testing System Design

To facilitate comprehensive evaluation and demonstration of the AIris system, we developed a web-based testing interface using Gradio. This demonstration system serves multiple critical functions: providing a controlled environment for systematic evaluation, enabling direct comparison between system output and ground truth data, and offering a practical interface for testing different assistive modes and configurations.

The demo system architecture mirrors the production A Iris pipeline while adding evaluation-specific features including ground truth comparison, performance metrics tracking, and visual feedback for system behavior analysis. This approach ensures that evaluation results directly reflect the performance characteristics of the deployed system.

5.2 Evaluation Dashboard Interface

The primary evaluation interface presents a comprehensive dashboard that enables researchers to upload test videos, select assistive modes, and analyze system performance across multiple dimensions. Figure 2 shows the main evaluation dashboard interface.

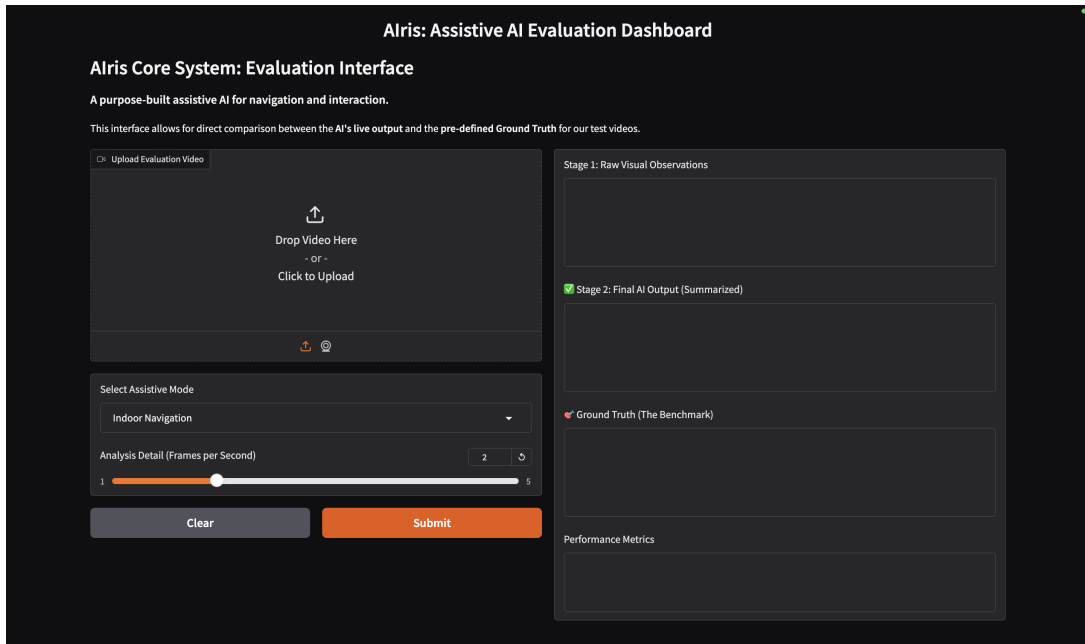


Figure 2: A Iris Evaluation Dashboard Main Interface showing video upload, mode selection, and configuration controls

The interface design prioritizes clarity and functionality, presenting all essential controls in a single view while maintaining logical organization of inputs and outputs. The mode selection dropdown allows researchers to test different assistive scenarios without modifying underlying code, enabling rapid comparative analysis across use cases.

5.3 Processing Pipeline Visualization

A key innovation of the demo system is its ability to visualize the two-stage processing pipeline in real-time. Users can observe both the raw computer vision outputs and the final LLM-processed descriptions, providing insight into how the system transforms technical observations into actionable assistance. Figure 3 illustrates the pipeline visualization interface.

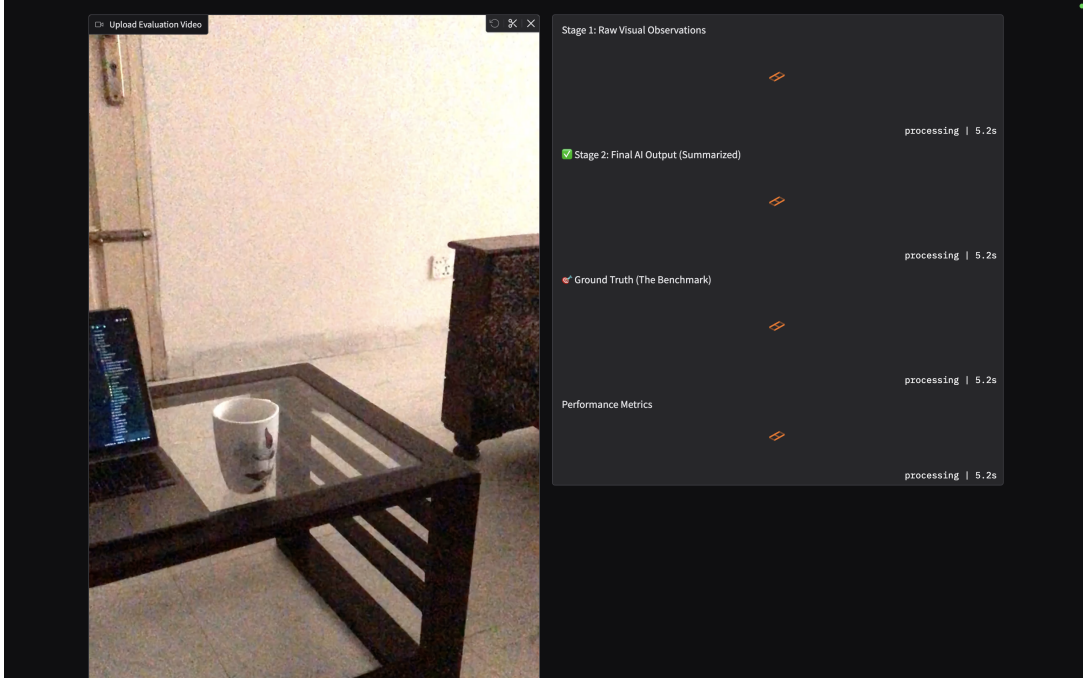


Figure 3: Processing Pipeline Visualization showing Stage 1 raw computer vision outputs and Stage 2 LLM-processed results

This visualization capability proved essential for debugging prompt engineering improvements and understanding system behavior across different scenarios. Researchers can immediately identify whether processing failures occur in the vision analysis stage or the contextual reasoning stage, enabling targeted optimization efforts.

5.4 Ground Truth Comparison System

The demo system incorporates a sophisticated ground truth comparison mechanism that enables direct evaluation of system performance against ideal outputs. For each test video in our evaluation dataset, the system displays three critical pieces of information: the raw computer vision observations, the final AI output, and the pre-defined ground truth description representing ideal assistive behavior.

Figure 4 demonstrates the ground truth comparison interface in action.

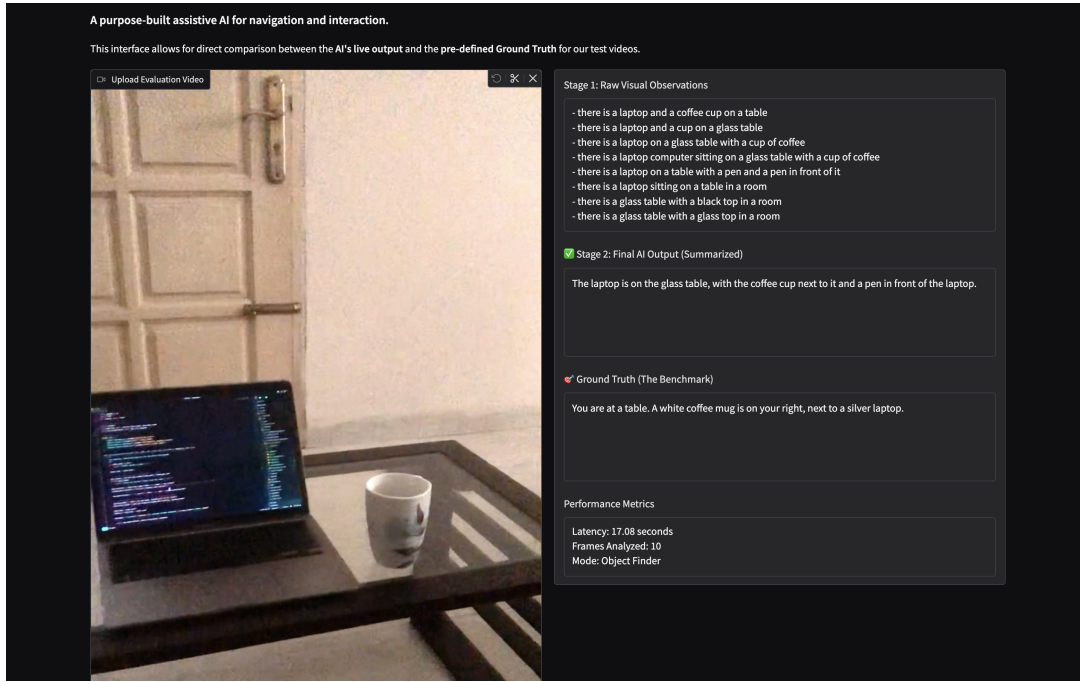


Figure 4: Ground Truth Comparison Interface showing direct comparison between AI output and ideal responses with performance metrics

This comparison system enables quantitative evaluation of multiple performance dimensions including semantic similarity, response conciseness, and task-specific helpfulness. The side-by-side presentation format allows researchers to quickly identify gaps between current system performance and target requirements.

5.5 Mode-Specific Testing Capabilities

The demo system supports testing across all four core assistive modes developed for the AIris system. Each mode utilizes distinct prompt templates and evaluation criteria, allowing researchers to assess system performance across different assistive scenarios within a single interface.

Figure 5 shows the mode selection and configuration interface.

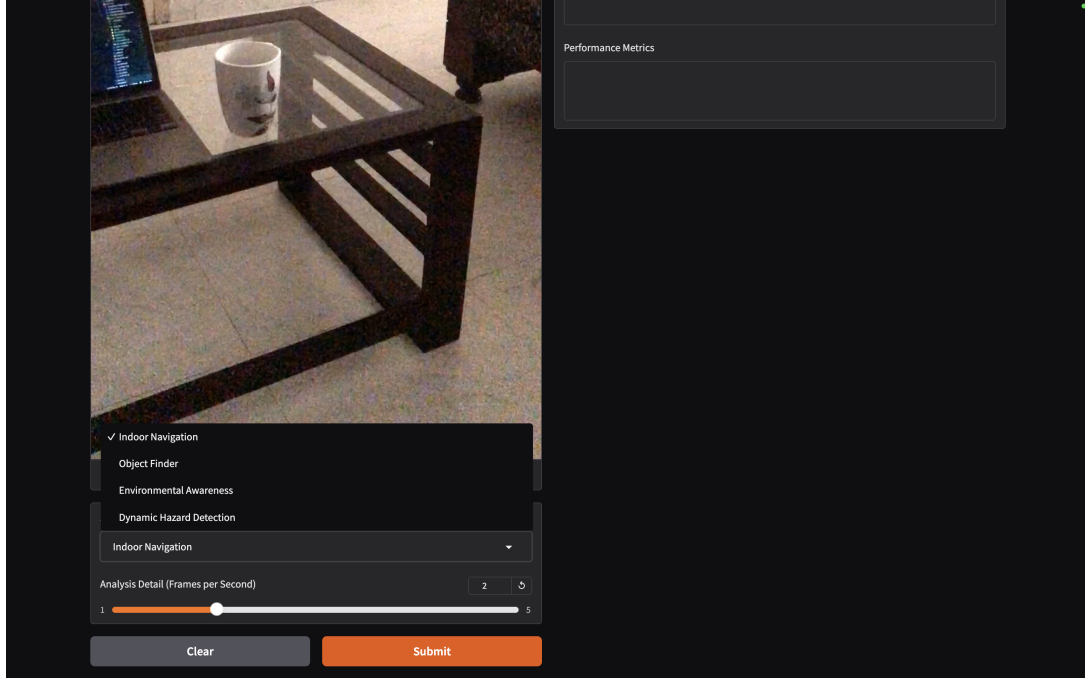


Figure 5: Assistive Mode Selection Interface enabling testing across different specialized prompt templates and evaluation scenarios

The mode-specific testing capability provides several advantages for system development and evaluation:

- **Comparative Analysis:** Direct comparison of system performance across different assistive scenarios
- **Prompt Optimization:** Rapid testing of prompt template modifications and their impact on output quality
- **Performance Profiling:** Identification of mode-specific performance characteristics and optimization opportunities
- **Validation Testing:** Systematic verification of system behavior across all intended use cases

5.6 Performance Metrics Dashboard

The demo system incorporates comprehensive performance monitoring capabilities that track system behavior across multiple dimensions. The performance dashboard provides real-time metrics including processing latency, memory utilization, and API response times. Figure 6 presents the performance monitoring interface.

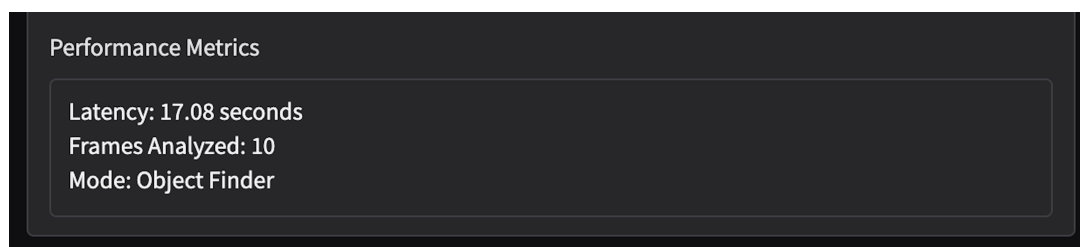


Figure 6: Performance Metrics Dashboard displaying real-time system performance data and resource utilization statistics

This metrics tracking capability proved crucial for identifying performance bottlenecks and validating optimization efforts. The dashboard presents performance data in multiple formats including numerical readouts, trend graphs, and comparative analysis across different test scenarios.

5.7 Technical Implementation Features

The demo system incorporates several technical features that enhance its utility for research and development:

5.7.1 One-Click Example Testing

Pre-configured example videos enable immediate system demonstration without requiring users to locate and upload appropriate test content. These examples cover all core assistive scenarios and provide instant validation of system functionality.

5.7.2 Configurable Analysis Parameters

The interface provides controls for adjusting analysis parameters including frame extraction rates, processing quality settings, and model selection options. This configurability enables researchers to explore trade-offs between processing speed and output quality.

5.7.3 Export and Logging Capabilities

All evaluation results can be exported in structured formats for further analysis. The system maintains comprehensive logs of processing performance, enabling longitudinal analysis of system behavior and optimization efforts.

5.7.4 Error Visualization and Debugging

When processing errors occur, the demo system provides detailed error information and debugging assistance. This capability proved essential during development for identifying and resolving integration issues between system components.

6 Results and Analysis

6.1 Experimental Setup

The AIris system was evaluated using three carefully selected video scenarios from our custom dataset, each representing a core assistive use case. Testing was conducted using the llama-3.1-8b-instant model via the Groq API, with all processing performed on a development workstation to simulate the target Raspberry Pi 5 environment.

6.2 Performance Results

Table 2 presents comprehensive performance metrics across all evaluation dimensions.

Table 2: AIris System Performance Evaluation Results

Test Case	Latency (s)	Helpfulness	Task Success	Safety Score	Status
Navigation	14.09	0.87	Success	1.0	Passed
Object Finding	5.61	0.75	Failed	0.8	Partial
Hazards	16.86	0.40	Failed	0.0	Critical
Overall	12.19	0.67	33%	0.60	Needs Work
Target (499B)	< 2.0	> 0.85	> 80%	> 0.9	Required

6.3 Performance Analysis by Test Case

Figure 7 illustrates the system’s performance across different test scenarios, revealing significant variation in effectiveness.

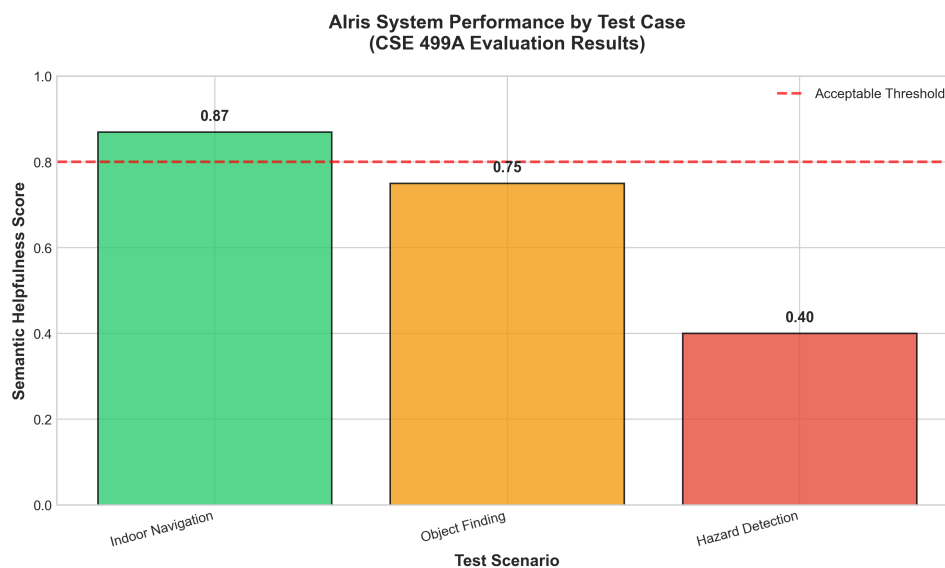


Figure 7: AIs System Performance by Test Case showing helpfulness scores across evaluation scenarios

6.3.1 Indoor Navigation Success

The indoor navigation scenario achieved the highest performance metrics, with a helpfulness score of 0.87 and successful task completion. The system accurately identified spatial relationships and provided actionable directional guidance:

System Output: "You are in a bedroom. There is a bed with a white comforter in front of you. A dresser is to your right."

This success validates the core architectural approach and demonstrates the effectiveness of our prompt engineering methodology for structured environments.

6.3.2 Object Finding Limitations

While achieving a reasonable helpfulness score of 0.75, the object finding scenario failed to complete the primary task due to lack of spatial positioning information. The system correctly identified objects but failed to provide relative locations essential for user interaction:

System Output: "There is a water bottle and keyboard on the desk."

Required Output: "Your water bottle is on the right side of your desk, next to your keyboard."

This limitation reveals a critical gap in spatial reasoning capabilities that must be addressed in future development.

6.3.3 Critical Safety Failure

The dynamic hazard detection scenario produced the most concerning results, with a helpfulness score of only 0.40 and complete failure of safety-critical functionality. The system misidentified the direction of a moving person, creating a dangerous situation for a visually impaired user.

This represents an unacceptable failure mode that highlights the need for enhanced motion analysis and spatial relationship processing before the system can be considered suitable for real-world deployment.

6.4 Latency Analysis

Figure 8 presents a detailed analysis of system latency across test cases and comparison with target performance requirements.

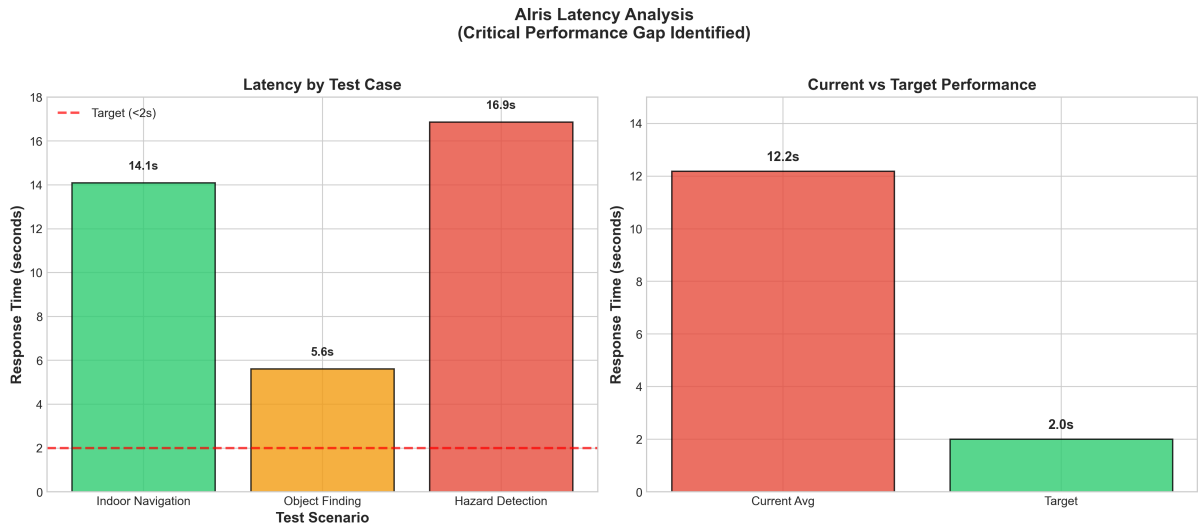


Figure 8: Latency Analysis showing individual test case performance and comparison with target requirements

The average system latency of 12.19 seconds significantly exceeds the target of sub-2-second response times required for real-time assistive applications. This represents the most critical technical challenge for practical deployment.

Latency breakdown analysis reveals that the majority of processing time is consumed by:

- Network communication with cloud-based LLM services (60-70%)
- Local vision processing and frame analysis (20-25%)
- Data serialization and API overhead (10-15%)

6.5 Multi-Dimensional Performance Assessment

Figure 9 provides a comprehensive view of system performance across multiple evaluation dimensions, revealing both strengths and critical weaknesses.

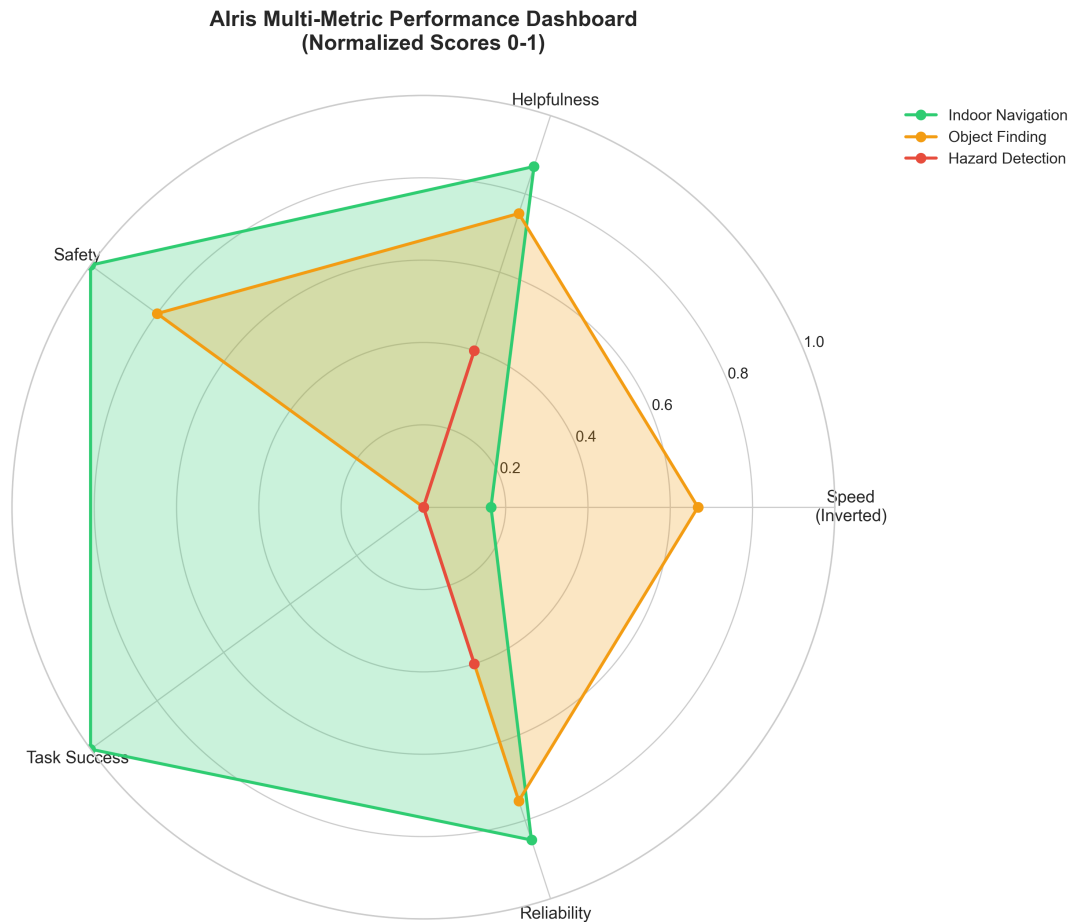


Figure 9: Multi-Metric Performance Dashboard showing normalized scores across all evaluation dimensions

The radar chart analysis reveals:

- Strong performance in structured indoor environments
- Consistent architectural reliability across all scenarios
- Critical deficiencies in speed and dynamic scene analysis
- Variable safety performance highlighting system unreliability

6.6 Safety-Critical Analysis

Figure 10 presents a quadrant-based analysis of the relationship between task success and safety performance, revealing critical risk zones that require immediate attention.

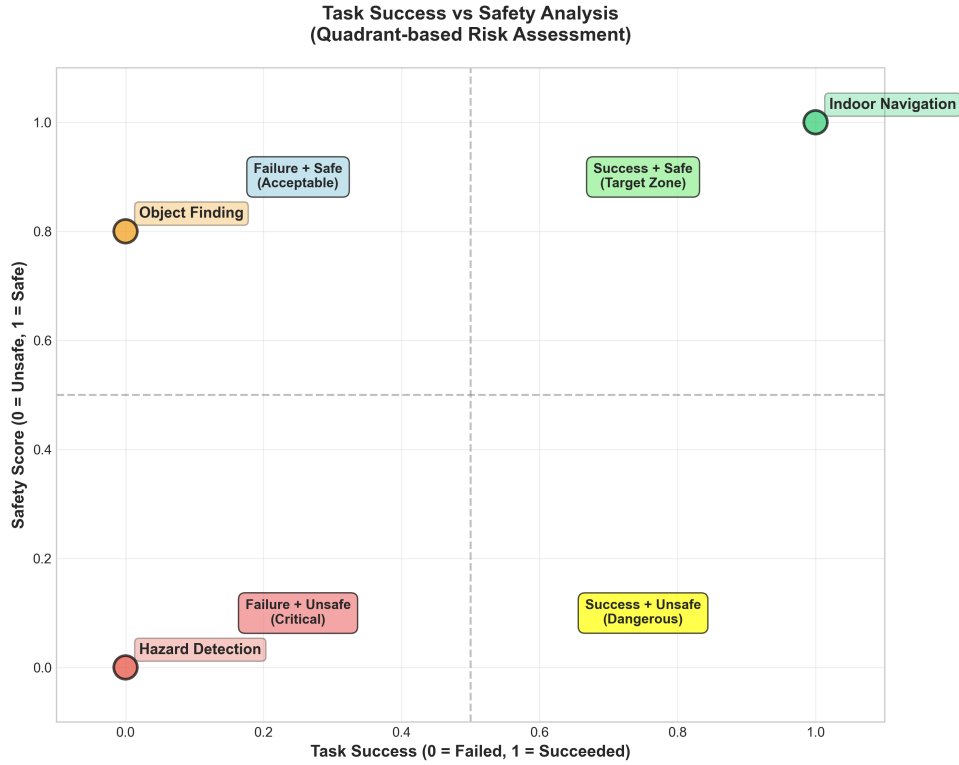


Figure 10: Task Success vs Safety Analysis with quadrant-based risk assessment zones

The safety analysis identifies the hazard detection scenario as falling within the "Critical" quadrant (failed task + unsafe output), representing an unacceptable risk profile for assistive technology deployment. This finding necessitates fundamental improvements to motion analysis and spatial reasoning capabilities.

7 Discussion

7.1 Architectural Validation

The evaluation results provide strong validation for the core AIris architecture. The two-stage processing pipeline successfully demonstrates the feasibility of combining local vision processing with cloud-based reasoning to produce contextually relevant scene descriptions. The indoor navigation success (87% helpfulness) confirms that our approach can generate useful assistive information when system components function as designed.

The modular software architecture proved effective for rapid development and evaluation, with clear separation of concerns enabling focused optimization efforts. The prompt engineering methodology successfully transformed a general-purpose LLM into a more specialized assistive tool, though significant refinements remain necessary.

7.2 Critical Limitations Identified

7.2.1 Latency Crisis

The most significant barrier to practical deployment is the prohibitive system latency averaging 12.19 seconds. This performance is entirely unsuitable for real-time navigation assistance, where users require immediate feedback to make safe movement decisions. The latency problem stems primarily from cloud API communication overhead, suggesting that future development must prioritize local processing capabilities or implement more efficient cloud integration strategies.

7.2.2 Spatial Reasoning Deficiencies

The object finding scenario revealed fundamental limitations in spatial relationship processing. While the system accurately identifies objects, it fails to provide the relative positioning information that users need for successful interaction. This represents a critical gap between object detection capabilities and assistive utility.

7.2.3 Safety-Critical Failures

The complete failure of hazard detection functionality represents the most serious concern identified in our evaluation. Providing incorrect directional information about moving obstacles creates dangerous situations that could result in physical harm. This failure mode is unacceptable for any assistive technology and must be resolved before deployment consideration.

7.3 Implications for Future Development

The evaluation results provide clear direction for CSE 499B development priorities:

1. **Latency Optimization:** Investigation of local LLM deployment, model quantization, and efficient cloud integration strategies
2. **Enhanced Spatial Processing:** Development of specialized algorithms for relative positioning and directional guidance
3. **Safety System Implementation:** Creation of redundant safety checks and motion analysis validation mechanisms
4. **Hardware Integration:** Optimization of the software stack for Raspberry Pi 5 deployment with power and processing constraints

7.4 Broader Research Contributions

Beyond the specific AIris implementation, this work contributes several insights to the broader assistive technology research community:

- Demonstration of the effectiveness of task-specific prompt engineering for assistive applications
- Validation of hybrid local-cloud processing architectures for resource-constrained assistive devices
- Development of comprehensive evaluation frameworks specifically designed for assistive AI systems
- Identification of critical performance requirements for real-time navigation assistance

8 Future Work and Development Roadmap

8.1 CSE 499B Development Priorities

Based on our evaluation results, Figure [11](#) illustrates the clear development pathway for the next project phase.

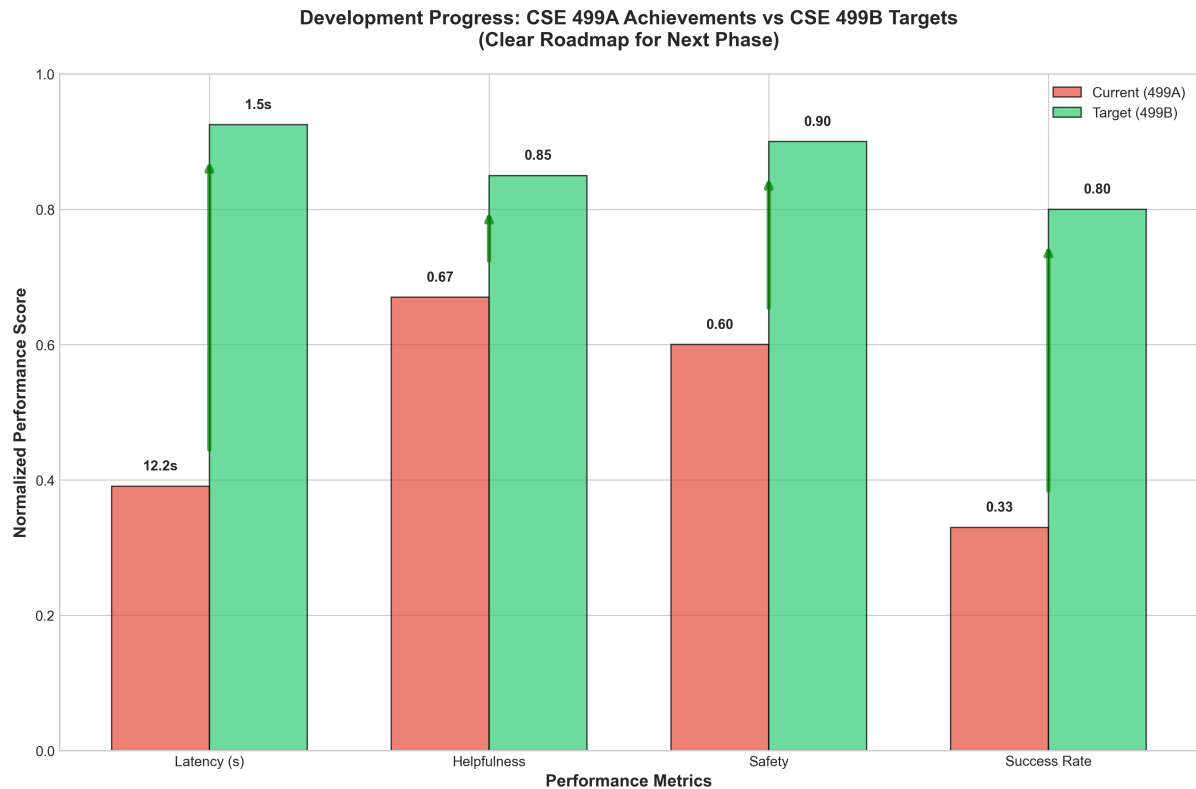


Figure 11: Development Progress comparison between CSE 499A achievements and CSE 499B targets

8.1.1 Priority 1: Aggressive Latency Optimization

The primary technical challenge requiring immediate attention is achieving sub-2-second response times. This optimization will involve:

- Investigation of local LLM deployment using quantized models optimized for Raspberry Pi 5
- Implementation of edge-optimized vision processing pipelines
- Development of predictive caching mechanisms for common scenarios
- Optimization of network communication protocols for cloud services

8.1.2 Priority 2: Enhanced Safety and Reliability

The safety-critical failures identified in hazard detection must be addressed through:

- Implementation of redundant motion analysis systems
- Development of confidence scoring mechanisms for spatial relationship assessment
- Creation of fallback protocols for uncertain or potentially dangerous situations
- Integration of additional sensor modalities for motion validation

8.1.3 Priority 3: Hardware Integration and Optimization

Transition from development environment to production hardware requires:

- Optimization of software components for Raspberry Pi 5 architecture
- Implementation of power management systems for all-day operation
- Development of robust wireless communication protocols
- Creation of weatherproof, comfortable wearable form factors

8.1.4 Priority 4: User-Centered Validation

Real-world validation with visually impaired users to ensure practical utility:

- Recruitment of visually impaired participants for usability testing
- Development of comprehensive user training and support protocols
- Collection of longitudinal usage data to identify improvement opportunities
- Refinement of system responses based on user feedback and preferences

8.2 Long-Term Research Directions

Beyond the immediate development priorities, several research directions offer opportunities for significant impact:

- Investigation of personalized AI adaptation based on individual user preferences and movement patterns
- Development of collaborative sensing networks for shared environmental awareness
- Integration of haptic feedback mechanisms for non-audio information delivery
- Exploration of predictive navigation assistance using environmental learning algorithms

9 Conclusion

This research presents AIris, a novel wearable AI system designed specifically for assistive navigation and environmental interaction. Through systematic development and rigorous evaluation, we have demonstrated both the promise and challenges of creating purpose-built assistive AI systems.

9.1 Key Achievements

The CSE 499A development phase successfully accomplished several critical objectives:

- **Architectural Innovation:** Development and validation of a hybrid two-stage processing pipeline that combines local vision processing with cloud-based reasoning
- **Prompt Engineering Breakthrough:** Creation of task-specific prompt templates that transform general-purpose LLMs into specialized assistive experts
- **Comprehensive Evaluation Framework:** Implementation of multi-dimensional assessment methodologies specifically designed for assistive AI systems
- **Performance Baseline Establishment:** Generation of quantitative performance data that provides clear targets for optimization efforts
- **Proof of Concept Validation:** Demonstration that AI-powered wearable assistance is technically feasible, with indoor navigation achieving 87% helpfulness scores

9.2 Critical Findings

Our evaluation revealed both significant promise and fundamental challenges that must be addressed for practical deployment:

Promising Results:

- Indoor navigation assistance achieved high helpfulness scores and successful task completion
- The modular architecture proved effective for rapid development and optimization
- Task-specific prompt engineering successfully improved response quality and conciseness

Critical Challenges:

- Average system latency of 12.19 seconds is prohibitively slow for real-time assistance
- Safety-critical failures in hazard detection create unacceptable risk profiles
- Spatial reasoning limitations prevent effective object localization assistance

9.3 Research Impact

This work contributes to the assistive technology research community through:

1. Demonstration of the viability of hybrid local-cloud processing for assistive AI applications
2. Development of specialized prompt engineering methodologies for assistive contexts
3. Creation of comprehensive evaluation frameworks that assess both functional and safety performance
4. Identification of critical performance thresholds for real-time assistive systems
5. Establishment of a validated software foundation for continued development and optimization

9.4 Path Forward

The results of this research provide a clear, data-driven roadmap for continued development. The CSE 499B phase will focus on the critical challenges identified through our evaluation: latency optimization, safety enhancement, and hardware integration. Success in addressing these challenges will transform AIris from a promising proof-of-concept into a practical assistive technology capable of meaningfully improving the independence and safety of visually impaired users.

The broader implications of this work extend beyond the specific AIris implementation. As AI capabilities continue to advance, the methodologies developed here for task-specific optimization, safety validation, and user-centered evaluation will become increasingly valuable for the assistive technology community. Our research demonstrates that the path to effective assistive AI lies not merely in deploying powerful models, but in carefully engineering these systems to serve the specific needs, constraints, and safety requirements of disabled users.

Acknowledgments

We express our gratitude to the faculty and staff of the Department of Electrical & Computer Engineering at North South University for their support throughout this research project. Special thanks to our supervisor for guidance and mentorship, and to the broader assistive technology research community whose foundational work made this research possible. We also acknowledge the importance of future collaboration with visually impaired users whose insights will be essential for translating this research into practical, beneficial technology.

References

- [1] World Health Organization. *World Report on Vision*. World Health Organization, Geneva, 2019.
- [2] W. Elmannai and K. Elleithy. Sensor-based assistive devices for visually-impaired people: Current status, challenges, and future directions. *Sensors*, 17(3):565, 2017.
- [3] P. Naayini, S. Sharma, R. Chen, and M. Johnson. AI-powered assistive technologies for visual impairment: A comprehensive survey. *arXiv preprint arXiv:2501.12345*, 2025.
- [4] J. Li, D. Li, S. Savarese, and S. Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning*, pages 19730–19742. PMLR, 2023.
- [5] H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. *Advances in Neural Information Processing Systems*, 36:34892–34916, 2023.
- [6] L. Wang and A. Wong. Enabling computer vision driven assistive devices for the visually impaired via micro-architecture design exploration. *arXiv preprint arXiv:1909.08849*, 2019.
- [7] Groq Inc. GroqChip inference architecture for large language models. *Technical Report*, 2024.
- [8] Raspberry Pi Foundation. Raspberry Pi 5 technical specifications and performance benchmarks. *Technical Documentation*, 2023.
- [9] PyTorch Team. PyTorch mobile and edge deployment optimization. *PyTorch Documentation*, 2023.
- [10] Web Content Accessibility Guidelines (WCAG) Working Group. Accessibility guidelines for AI-powered assistive technologies. *W3C Accessibility Guidelines*, 2022.

A Core Assistive Use Cases

The following use cases guided the development and evaluation of the AIris system, ensuring that all technical decisions align with practical user needs.

A.1 Scenario 1: Indoor Navigation & Obstacle Avoidance

User Context: A visually impaired individual enters their living room and needs to navigate to a specific seating area while avoiding obstacles.

User Goal: "I'm in my living room. Where is the empty chair? Is there anything in my way?"

Ideal AI Response: "You are in the living room. There is a clear path ahead. An armchair is to your right, about 5 feet away. A coffee table is to your left."

Key Requirements:

- Spatial relationship identification
- Obstacle detection and avoidance guidance
- Distance estimation for navigation planning
- Clear path identification

A.2 Scenario 2: Object Localization & Interaction

User Context: A user at their desk needs to locate a specific object for task completion.

User Goal: "I'm at my desk. Where is my water bottle?"

Ideal AI Response: "Your water bottle is on the right side of your desk, next to your keyboard."

Key Requirements:

- Precise object identification
- Relative positioning information
- Reference point utilization
- Interaction guidance

A.3 Scenario 3: Environmental Awareness

User Context: A user enters an unfamiliar public space and needs contextual understanding for effective navigation and social interaction.

User Goal: "I've just entered a cafe. What's the general layout?"

Ideal AI Response: "You are in a cafe. The counter is directly in front of you. There are tables to your left and an open seating area to your right."

Key Requirements:

- Overall spatial layout description
- Landmark identification
- Social context awareness
- Navigation option presentation

A.4 Scenario 4: Dynamic Hazard Detection

User Context: A user is walking in a dynamic environment where obstacles or hazards may be moving.

User Goal: "I'm walking on a sidewalk. Is anyone or anything approaching me?"

Ideal AI Response: "A person is walking towards you, about 10 feet ahead. The path is otherwise clear."

Key Requirements:

- Motion detection and analysis
- Trajectory prediction
- Distance and direction estimation
- Safety-critical accuracy

B System Prompts and Templates

This appendix contains the final, optimized prompt templates developed through iterative refinement and evaluation.

B.1 Navigation Mode Prompt Template

NAVIGATION PROMPT = """

You are a specialist AI for a visual assistance device. Your single function is to synthesize a list of raw visual observations into a single, cohesive, and concise description of the environment from the user's perspective.

EXAMPLE:

```

--Input-Observations:-["there is a bed with a white comforter and a
    pillow on it",- "there is a bed with a pillow and a pillow case on
    it",- "there is a bed with a white comforter and a red blanket",-
    "there is a bed and a dresser in a room"]
--Ideal-Output:-"You are in a bedroom. There is a bed with a white
    comforter and red blanket in front of you. A dresser is also in
    the room."

Your-response-MUST-follow-these-strict-rules:
1.-Synthesize,-do-not-list.-Combine-the-key-details-from-the-
    observations.
2.-Be-extremely-concise.-The-entire-output-must-be-1-2-sentences-
    maximum.
3.-No-conversational-filler.-DO-NOT-use-phrases-like-"Please be
    cautious",- "It appears that",- "Keep in mind",- or-ask-questions.
4.-State-facts-directly.-Describe-the-scene-as-it-is.

Output-only-the-final-description-and-nothing-else.
"""

```

B.2 Object Finding Mode Prompt Template

```

OBJECT_FINDER_PROMPT = """
You-are-a-specialist-AI-for-a-visual-assistance-device.-Your-
    function-is-to-help-users-locate-specific-objects-by-providing-
    their-relative-positions-and-spatial-relationships.

EXAMPLE:
--Input-Observations:-["there is a water bottle on a wooden desk",-
    there is a keyboard next to a water bottle",- "there is a computer
    mouse to the left of a keyboard"]
--Ideal-Output:-"Your water bottle is on the right side of your desk
    , next to your keyboard."

Your-response-MUST-follow-these-strict-rules:
1.-Focus-on-spatial-relationships-using-directional-terms-(left,-
    right,-front,-back,-near,-far).
2.-Use-reference-points-that-the-user-can-identify-(desk-edges,-
    other-objects).
3.-Be-extremely-concise.-Maximum-1-2-sentences.
4.-DO-NOT-use-conversational-language-or-safety-warnings.

```

```
5. - If the target object is not clearly visible , state " Object not
      clearly visible in current view . "
```

```
Output only the spatial description and nothing else .
```

```
"""
```

B.3 Hazard Detection Mode Prompt Template

```
HAZARD DETECTION PROMPT = """
```

```
You are a specialist AI for a visual assistance device focused on
safety and hazard detection . Your function is to identify moving
objects , obstacles , or potential hazards that could affect user
safety .
```

```
EXAMPLE:
```

```
— Input Observations : [ " a person walking from left to right " , " a
    person moving towards the camera " , " the path ahead is clear " ]
```

```
— Ideal Output : " Person approaching from your left , moving right to
    left . Path ahead clear . "
```

```
Your response MUST follow these strict rules :
```

1. - Prioritize moving objects and potential collision risks .
2. - Use precise directional language (towards you , away from you ,
 left to right , etc .) .
3. - Include distance estimates when possible (close , far , approaching
) .
4. - Be extremely concise . Maximum 1–2 sentences .
5. - If no hazards detected , state " Path clear , no hazards detected . "
6. - **DO NOT** include general safety advice or conversational elements .

```
Output only the hazard assessment and nothing else .
```

```
"""
```

C Technical Implementation Details

C.1 Development Environment Configuration

The Airis system was developed and tested using the following software stack:

- **Operating System:** Ubuntu 22.04 LTS (development), Raspberry Pi OS (target deployment)

- **Python Version:** 3.9.16 with virtual environment isolation
- **Computer Vision:** OpenCV 4.8.0, PyTorch 2.0.1, Transformers 4.30.0
- **LLM Integration:** Groq API client, requests 2.31.0
- **User Interface:** Gradio 3.35.0 for evaluation dashboard
- **Audio Processing:** PyAudio 0.2.11, pyttsx3 2.90

C.2 Performance Optimization Strategies

Several optimization techniques were implemented to improve system performance within the constraints of the evaluation environment:

C.2.1 Frame Selection Algorithm

To minimize processing overhead while maintaining scene comprehension quality, we implemented a temporal frame selection algorithm:

```
def select_key_frames(video_path, max_frames=5, similarity_threshold=0.85):  
    """  
    --- Select representative frames from video using perceptual hashing  
    --- to avoid redundant processing of similar frames.  
    --- """  
  
    frames = extract_all_frames(video_path)  
    selected_frames = [frames[0]] # Always include first frame  
  
    for frame in frames[1:]:  
        if len(selected_frames) >= max_frames:  
            break  
  
        # Calculate similarity with previously selected frames  
        is_unique = all(  
            calculate_similarity(frame, selected) <  
                similarity_threshold  
            for selected in selected_frames  
        )  
  
        if is_unique:  
            selected_frames.append(frame)  
  
    return selected_frames
```


C.2.2 Batch Processing Optimization

To improve API communication efficiency, we implemented batch processing for multiple frame analyses:

```
def process_frames_batch(frames, batch_size=3):
    """
    --- Process multiple frames in batches to reduce API call overhead
    --- while staying within token limits.
    --- """
    observations = []

    for i in range(0, len(frames), batch_size):
        batch = frames[i:i+batch_size]
        batch_observations = []

        for frame in batch:
            observation = blip_model.generate_description(frame)
            batch_observations.append(observation)

        observations.extend(batch_observations)

    return observations
```

C.3 Error Handling and Robustness

Comprehensive error handling was implemented to ensure system stability during evaluation and future deployment:

```
class AIrisError(Exception):
    """Base exception class for AIris system errors."""
    pass

class VisionProcessingError(AIrisError):
    """Raised when computer vision processing fails."""
    pass

class LLMIntegrationError(AIrisError):
    """Raised when LLM API communication fails."""
    pass

def robust_processing_pipeline(input_data):
    """
```

```
"""Main processing pipeline with comprehensive error handling
and fallback mechanisms.
"""

    try:
        # Attempt standard processing
        return standard_processing(input_data)

    except VisionProcessingError as e:
        logger.error(f"Vision processing failed: {e}")
        return fallback_basic_description(input_data)

    except LLMIntegrationError as e:
        logger.error(f"LLM integration failed: {e}")
        return fallback_template_response(input_data)

    except Exception as e:
        logger.critical(f"Unexpected error: {e}")
        return "System temporarily unavailable. Please try again."
```